

南京林业大学试卷

课程 单片机技术基础 (A) 2018-2019 学年第 2 学期

题号	一	二	三	四	五	六	七	八	九	十	十一	十二	十三	十四	十五	总分
得分																

金市刺客

20508653

一 填空题 (每空 1 分, 共 30 分)

1. 设二进制数 $A=10101101$, $B=01110110$, 则逻辑运算 $A \vee B =$ _____, $A \wedge B =$ _____, $A \oplus B =$ _____。

2. 通常单片机有两种复位操作, 即 上电复位 和 按键复位。复位后 PC 值为 0000H, SP 值为 07H, 通用寄存器的当前寄存器组为 00 组, 该组寄存器的地址范围是从 00H 到 07H。

3. P0~P3 口在作输入口使用时, 应先向口锁存器写入 高电平, 方能准确无误的读取引脚上的 高电平 信号; 而 P0 口用作输出口时, 必须外接 上拉电阻 才能输出 高电平。

4. MCS-51 片内 20H-2FH 范围内的数据存储器, 既可以字节寻址又可以位寻址。

5. 89S51 单片机的定时器/计数器工作模式寄存器名是 TMOD, 其中 C/T 的作用是 选择工作方式。当 M1M0=00 时, 其计数器的长度是 13 位, 对应的定时时间计算表达式为: 定时时间 = (2^13 - 初值) × 机器周期。

6. MCS-51 单片机的串行口在工作方式 0 下, 是把串行口作为 同步移位寄存器 来使用。这样, 在串入并出移位寄存器的配合下, 就可以把串行口作为 并行输出口 使用。 (并入串出: 并行输入口)

7. LED 显示器按内部发光二极管的公共点连接方法不同, 有 共阴极 和 共阳极 分。LED 显示有 动态 和 静态 两种方式。

8. 单片机的键盘结构有两种, 分别是 独立连接 和 矩阵连接。

9. 外中断请求标志位是 IE0 和 IE1。

选择题 (每题 2 分, 共 20 分)

1. 8 位二进制数所能表示的最大无符号数是 (A)
A. 255 B. 256 C. 128 D. 127
2. MCS-51 单片机的四个并行 I/O 中, 其驱动能力最强的是: (A)
A. P0 口 B. P1 口
C. P2 口 D. P3 口
3. 单片机的 ALE 引脚是 (B)
A. 输出高电平 B. 输出矩形脉冲, 频率为 f_{osc} 的 1/6;
C. 输出低电平 D. 输出矩形脉冲, 频率为 f_{osc} 的 1/2;
4. MCS-51 单片机复位后, 从下列那个单元开始取指令: (C)
A. 0003H B. 000BH
C. 0000H D. 0013H
5. 在相对寻址方式中, “相对” 两字是指相对于 (C)
A. 地址偏移量 rel B. 当前指令的首地址
C. 下一条指令的首地址 D. DPTR 值
6. 对定时器控制寄存器 TCON 中的 IT0 和 IT1 清 0 后, 则外部中断请求信号方式为 (A)
A. 低电平有效 B. 高电平有效
C. 脉冲上跳沿有效 D. 脉冲后沿负跳有效。
7. MCS-51 单片机的定时器 T0 用作计数方式时是 (C)
A. 由内部时钟频率定时, 一个时钟周期加 1;
B. 由内部时钟频率定时, 一个机器周期加 1;
C. 由外部计数脉冲计数, 下降沿加 1;
D. 由外部计数脉冲计数, 一个机器周期加 1;
8. MCS-51 单片机串行接收数据的次序时下述的顺序 (B)
(1) 接收完一帧数据后, 硬件自动将 SCON 的 RI 置 1;
(2) 用软件将 RI 清零; (3) 接收到的数据由 SBUF 读出
(4) 置 SCON 的 REN 为 1, 外部数据由 RXD(P3.0)输入;
A. (1) (2) (3) (4) B. (4) (1) (2) (3)
C. (4) (3) (1) (2) D. (3) (4) (1) (2)

9. N 位 LED 显示器采用动态显示方式时, 需要提供的 I/O 线总数是: (A)
A. $8+N$ B. $8 \times N$ C. N D. $2 \times N$

10. 某异步通讯接口的波特率为4800, 则该接口每秒钟传送()。

- A. 4800 位 B. 4800 字节 C. 9600 位 D. 9600 字节

A

三 问答题 (每题 5 分, 共 20 分)

1. 89S51 单片机 ROM 空间中, 0000H~0023H 有什么用途? 用户应怎样合理安排?

答: 0000H~00023H 就是 80C51 系统专用单元, 其中 0000H 为 CPU 复位地址, 0003H~0023H 就是 5 个中断源中断服务程序入口地址, 用户不能安排其他内容。一般来讲, 从 0030H 以后, 用户可自由安排。

2. 定时器 0 工作于方式 2 的计数方式, 预置的计数初值为 156, 若通过引脚 T0 输入周期为 1ms 的脉冲, 则定时器 0 的定时时间为?

2. 方式 2 是自动重装初值 8 位计数器
计数值 = $2^8 - \text{初值} = 256 - 156 = 100$
定时时间 = 计数值 $\times$ 输入脉冲周期
= $100 \times 1\text{ms} = 100\text{ms}$

3. 寄存器 SBUF 有什么作用? 请画出串行口接收数据的时序图。

答: 串行缓冲寄存器 SBUF 有两个: 一个就是串行发送缓冲寄存器, 另一个就是串行接收缓冲寄存器, 用同一个特殊功能寄存器名 SBUF 与同一单元地址 99H。接收缓冲寄存器还具有双缓冲结构, 以避免在数据接收过程中出现帧重叠错误。

在完成串行初始化后, 发送时只需将发送数据输入 SBUF, CPU 将自动启动与完成串行数据得发送; 接收时 CPU 将自动把接收到得数据存入 SBUF, 用户只需从 SBUF 中读取接收数据。

4. 请说明关键字 data

存储器类型	描 述
data	直接寻址的片内 RAM 低 128B，访问速度快
bdata	片内 RAM 的可位寻址区（20H~2FH），允许字节和位混合访问
idata	间接寻址访问的片内 RAM，允许访问全部片内 RAM
pdata	用 Ri 间接访问的片外 RAM 的低 256B
xdata	用 DPTR 间接访问的片外 RAM，允许访问全部 64k 片外 RAM
code	程序存储器 ROM 64k 空间

四 编程题（每题 10 分，共 30 分）

1. 请编程实现把外 RAM 中 0200H 单元开始的 20 个字节数据，传送到内 RAM 从 40H 开始的区域中。

```
#include <reg52.h>
#include <intrins.h>
unsigned char xdata a[20] _at_ 0x0200;//片外 0200H 后的 20 个字节数据
unsigned char data b[20] _at_ 0x40;//片内 40H 后的 20 个字节数据
void main()
{
    int i=0;
    for(i=0;i<20;i++)
    {
        b[i]=a[i];
    }
    while(1)
    {
    }
}
```

2. 设晶振为 12MHZ，选用 T1 的工作方式 1 产生 1ms 定时，在 P1.1 口输出周期为 2ms 的方波（用中断法）。

```
#include<reg51.h>
sbit P1_1=P1^1;    //定义引脚
#define ui unsigned int
//周期为 0.2ms    a=65536-1000=55436=0xD8F0 （初值=2^n-定时时间/机器周期）
ui a = 0xD8F0;

void init(ui a){
    TMOD=0x10; //TMOD 初始化： T1 中断方式 1 定时
    TH1=a/256;  //设定初值高八位 D8
    //（寄存器是 8 位的，两个 8 位寄存器组成的 16 位计数器）
    TL1=a%256;  //设定初值低八位 F0
    IE=0x88;    //IE 初始化： EA=1 开总中断 ET1=1 开 T1 中断（IE 可以位寻址）
    TCON=0x40; //TCON 初始化： TR1=1 启动 T1（T1、T0 的溢出、运行）
}

timer1() interrupt 3{    //T1 中断函数
    P1_1=!P1_1;          //取反
    TH1=a/256;            //重载高八位计数值
    TL1=a%256;            //重载低八位计数值
}

void main(void){
    init(a);
    while(1);
}
```

3. 编写一个循环闪烁的程序。有 8 个发光二极管，循环依次点亮，当外中断 0 信号出现时，8 个发光管同时闪烁十次后，又开始循环依次点亮。请画出相关的单片机连接电路图。（10 分）

```
#include<reg51.h>
#define uchar unsigned char
bit ldelay=0;
uchar t=0;
timer0 interrupt 1
{
    t++;
    TH0=0x3c;
    TL0=0xb0;
    if(t==20)
        {ldelay=1;t=0;}
}
main
{
    uchar table[]={0xfe,0xfd,0xfb,0xf7,0xef,0xdf,0xbf,0x7f};
    int i=0;
    int j=1;
    TMOD=0x01;
    TH0=0x3c;
    TL0=0xb0;
    EA=1;
    ET0=1;
    TR0=1;
    while(1)
        if(ldelay)
            {
                ldelay=0;
                if(j)
                {
                    P2=table[i];
                    i++;
                    if(i==8) {i=6;j=0;}
                }
            }
        else
        {
            P2=table[i];
            i;
            if(i==1) {i=1;j=1;}
        }
    }
}
```

